Dispozitive și Aplicații Mobile

Cursul 3-4

Sumar

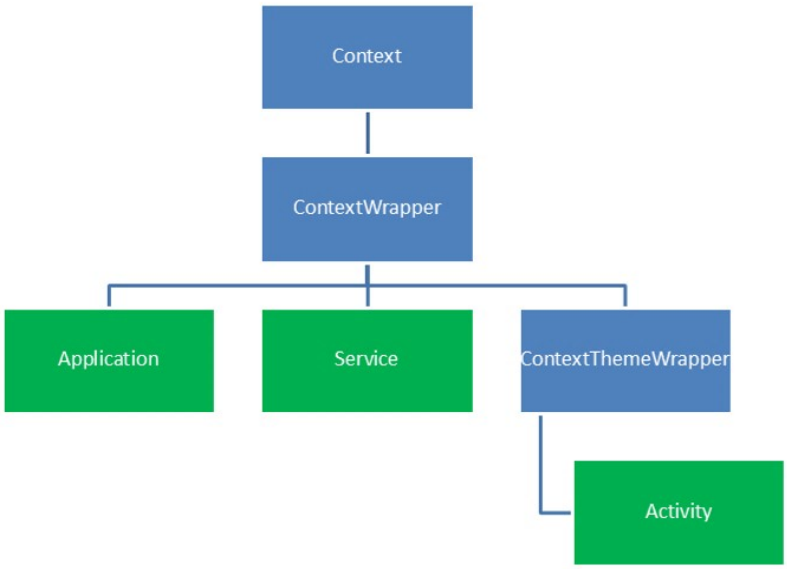
- Contextul și clasa aplicație
- Activități
- Resurse
- Interfața grafică în mod imperativ
  - Fișiere XML (machete), containere, componente vizuale
- Interfața grafică în mod procedural
- Tratarea evenimentelor

CONTEXTUL ȘI CLASA APLICAȚIE

Contextul

- Asigură accesul la mediul aplicației/componentei
- Utilizare
  - Crearea componentelor vizuale
  - Acces la resurse (referințe)
  - Lansarea de activități noi
  - Acces la servicii de sistem
  - Acces la obiectul de tip aplicație
  - Acces la baze de date și fișiere, inclusiv de preferințe
  - Administrare permisiuni
- Clasa abstractă *Context,* pachetul *android.content*

Contextul



Aplicația

- Lansată în execuție de utilizator
- Ciclu de viață propriu
- Sistemul poate încheia execuția aplicației

Clasa aplicație

- Accesul la setările și metodele aplicației
- Clasă derivată din **Application**
- Generată implicit
- Referință obiect aplicație: metoda **getApplication**()
- Poate fi creată o clasă utilizator prin derivare
  - De referit în fișierul **AndroidManifest.xml**!
- Are un context asociat
  - Disponibil pe toată durată rulării aplicației

Accesul la context

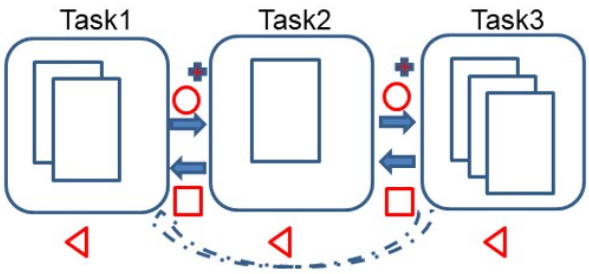
- La nivel de aplicație
  - **getApplicationContext()**
- La nivel de activitate/serviciu
  - obiectul curent (this)
- La nivel de control
  - Fiecare obiect grafic referă contextul activității asociate
  - **getContext()**

ACTIVITĂȚI

Activități

- Asociate ecranelor aplicației
- O aplicație poate avea una sau mai multe activități
- Derivate din clasa de bază android.app.**Activity**
- Stiva de activități
  - *Task*-uri

Stiva de activități



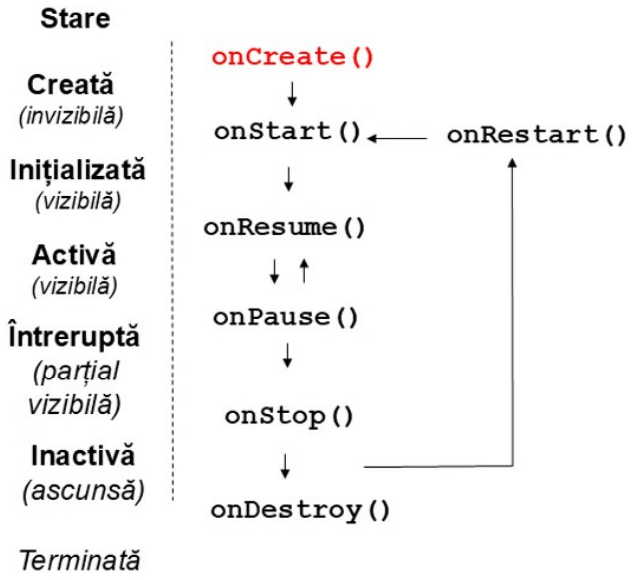
- Task n – stivă de activități
- Fiecare activitate nouă poate fi amplasată:
  - în stiva curentă
  - într-o nouă stivă

- **Legendă**
  - ✚ *lansarea unei noi activități din lista de aplicații recente (□) sau din lista de programe (după apăsarea butonului Home ○)*
  - ◁ *Tasta Back*

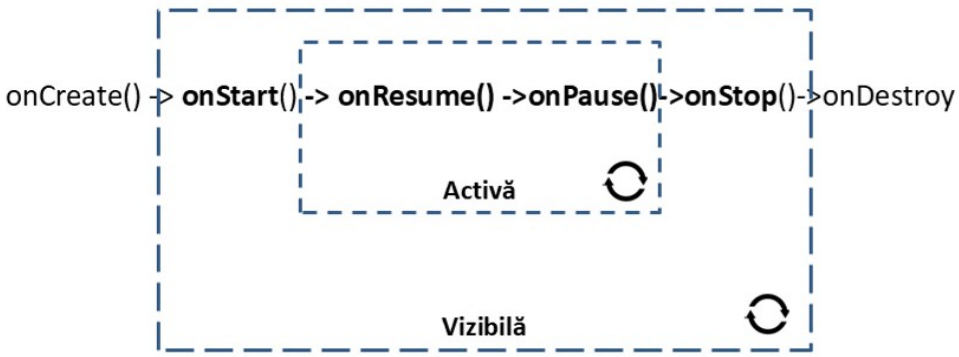
Activități

- Au o fereastră asociată
  - Reprezentarea interfeței grafice
- Dispun de un ciclu de viață
  - Mai multe stări
  - Metode cu apel invers
    - Invocate la trecerea într-o stare
    - Posibilitatea de salvare a stării activității (conținut, poziție componente vizuale, proprietăți etc.)

Activități: ciclul de viață



Activități: ciclul de viață



Activități

```
import android.app.Activity;
//import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class TestActivity extends Activity //AppCompatActivity
{
  @Override
  public void onCreate(Bundle stare) {
    super.onCreate(stare);
     //inițializare conținut vizual
     setContentView(view);
    //inițalizari activitatate
   }
}
```

Modificarea configurației

- Schimbarea orientării dispozitivului
  - rotire
- Activarea/dezactivarea tastaturii fizice
- Schimbare limbii utilizate
  - Setări regionale
- Activitățile sunt recreate pentru a corespunde noilor condiții

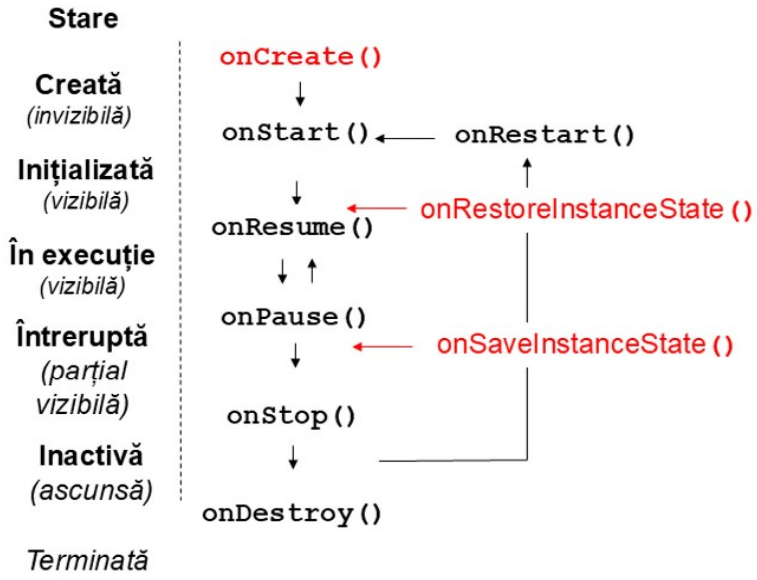
Clasa Bundle

- Permite stocarea și regăsirea de valori după o cheie de tip **String**
- Valorile stocate
  - Tipuri simple
  - Tipuri care implementează interfețele **Parcelable** sau **Serializable**
- Adăugare valori
  - **putString**(cheie, valoare_String)
  - **putInt** (cheie, valoare_int)
  - **putObject**(cheie, valoare_object)
  - **putLongArray**(cheie, valoare_long_array) *etc.*
- Regăsire valori
  - **getString**(cheie)
  - **getInt**(cheie)
  - **getObject**(cheie)
  - **getLongArray**(cheie) *etc.*

Salvarea/restaurarea stării

- Salvarea stării
  - **onSaveInstanceState**(Bundle stare)
    - Apelată la ieșirea forțată din aplicație (modificarea configurației)
- Restaurarea stării
  - **onRestoreInstanceState**(Bundle stare) *sau*
  - **onCreate**(Bundle stare)
    - *stare* poate fi null!

Activități: ciclul de viață (2)



Invocarea activităților (1)

- Automat, la lansarea aplicației
  - Filtrul de mesaje
    - action android:name="android.intent.action.MAIN"
- Metoda **startActivity() din** clasa Context
  - Parametrul: obiect de tip **Intent** asociat activității

Invocarea activităților (1) – cont.

- Obiectul de tip **Intent**
  - Contextul asociat (**Context**)
  - Clasa asociată activității (**Class<?>**)
- Exemplu (clasa asociată activității este *Activitate*)
  1. **Intent** i = new **Intent**(this, Activitate.class);
  2. **startActivity**(i);

Terminarea activităților

- Utilizatorul
  - Revenire (Back)
  - Închidere (Task)
- Din cod: apelul metodei **finish()**
- Sistemul

Proprietăți ale activităților în AndroidManifest.xml

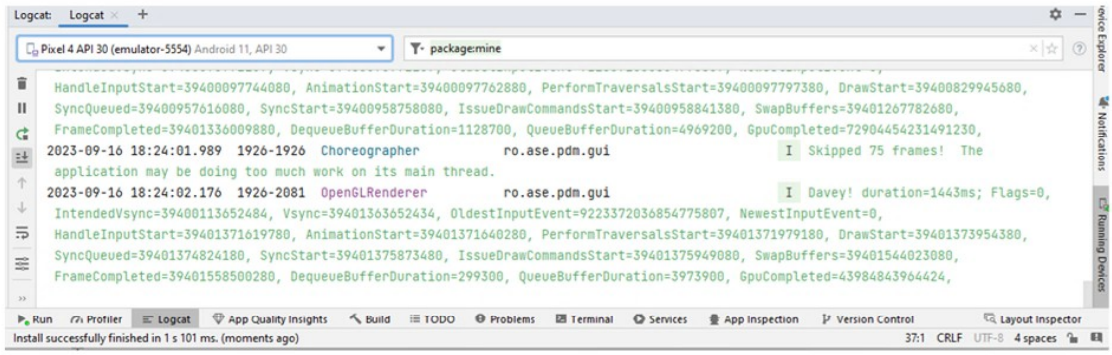
```
<activity
        android:name=".MainActivity"
        android:label="@string/app_name"
        android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

CONSOLA DE MESAJE (LOGCAT)

Consola de mesaje (Logcat)

- Afișează mesaje transmise din aplicații
  - Utilizator
  - Sistem
- Tipuri de mesaje
  - Avertizare (**w**)
  - Depanare (**d**)
  - Eroare (**e**)
  - Informare (**i**)
  - Informare detaliată (**v**)
  - Eroare excepțională (**wtf**)
- Suport căutare și filtrare

Consola de mesaje (Logcat)



Consola de mesaje (Logcat)

- Clasa android.util.**Log**
- Metode statice asociate tipurilor de mesaje:
  - **e**(), **w**(), **i**(), **d**(), **v**(), **wtf**()
- Parametri:
  - Identificator sursă mesaj (String)
    - Numele clasei, aplicației, activității etc.
    - Posibilitatea de filtrare
  - Mesajul care va fi afișat (String)
- Metoda statică generală
  - **println**()
  - În plus, primul parametru include tipul mesajului
    - **Log.ASSERT**, **Log.ERROR**, **Log.INFO** etc.

Consola de mesaje (Logcat)

- **Log.i**("Activitate1", "Mesaj de informare");
- **Log.println**(Log.***ASSERT***, "Activitate 1", "Aserțiune invalida!");

RESURSE

Tipuri de resurse

- Șiruri de caractere, culori, masive, stiluri, constante
- Machete interfață aplicație
- Imagini, pictograme, grafică vectorială
- Fișiere XML
- Fișiere de date
- Animații
- etc.

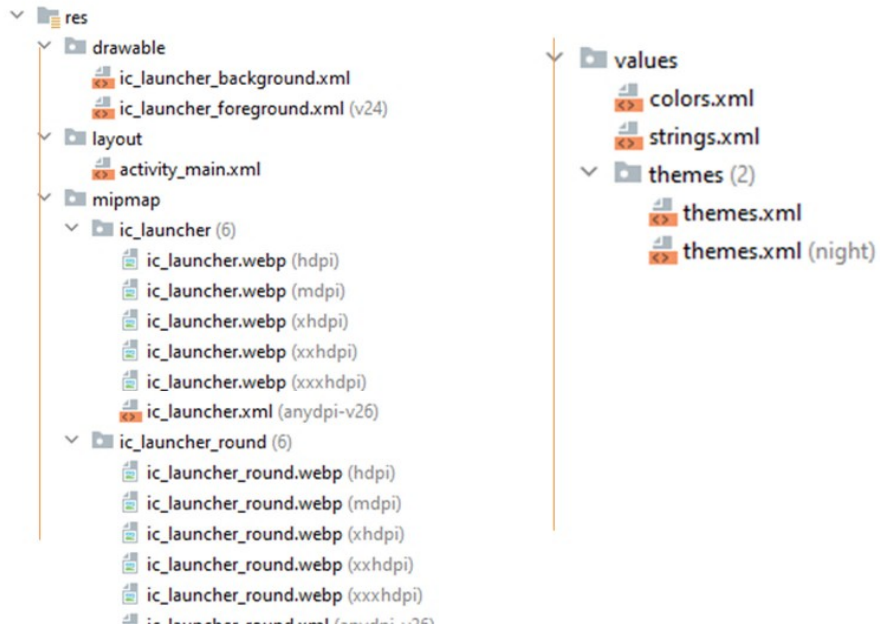
Resurse (1)

- Stocate în directorul **res**
- Fișiere XML sau fișiere specifice
- Grupate în directoare dedicate
- Identificatori asociați
  - Referire din fișiere XML de resurse/configurare
  - Referire din cod
- Definire resurse în funcție de configurație (orientare ecran, setări regionale, rezoluție etc.)

Directoare resurse

- res/values
  - Șiruri de caractere (*string*), culori (*color*), masive (*array*), *constante* (*dimen*), stiluri (style) etc.
- res/drawable
  - Imagini, grafică vectorială
- res/mipmap
  - Pictograme aplicație
- res/xml
  - Fișiere XML
- res/raw
  - Fișiere de date aplicație
- res/anim
  - Animații

Directoare resurse



Fișiere XML de resurse

```
<resources>
   <tip_res name="nume_res">Resursa</tip_res>
</resources>
```

Exemple de resurse

```
<resources>
 <string name="mesaj">Document nou</string>
</resources>

<resources>
    <color name="culoare_fundal">#00CCCC</color>
</resources>

<resources>
    <array name="optiuni">
            <item>zip</item>
            <item>rar</item>
            <item>7z</item>
    </array>
</resources>
```

Referirea resurselor

- Resursele pot fi accesate, pe baza unui identificator sau a numelui
  - din alte fișiere XML
  - din codul sursă Java/Kotlin
- Fiecare resursă, pe baza tipului și a numelui, are asociat un identificator în clasa generată **R**
  - clasa predefinită **android.R**

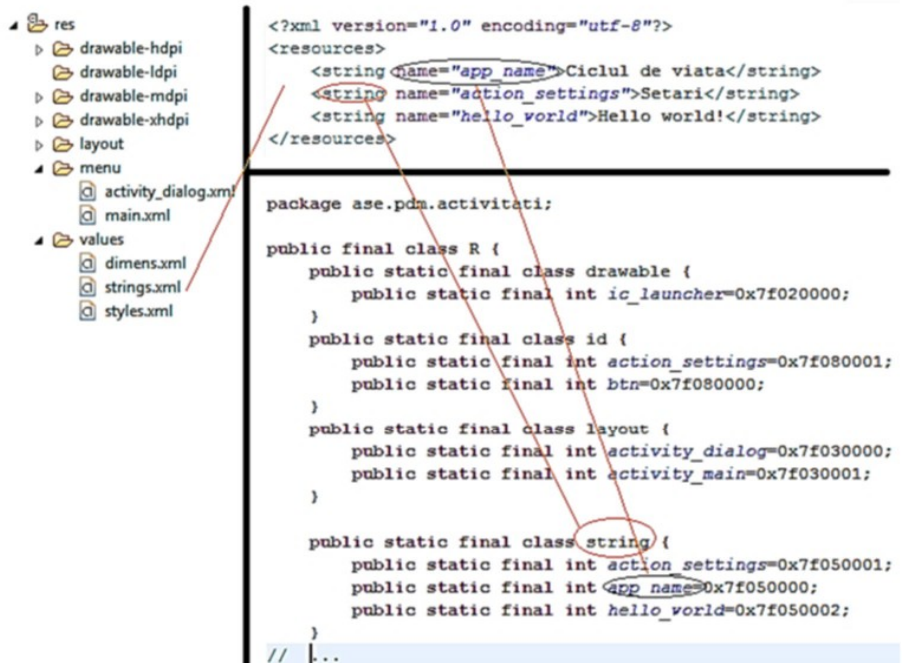
Referirea resurselor din fișiere XML

- Structura referinței
  - @*resursa*/nume
  - @android:*resursa*/nume
- Exemple
  - *@string/mesaj*
  - *@color/culoare_fundal*
  - *@array/optiuni*
  - *@id/id_element*
  - *@+id/id_text1*

Referirea resurselor din cod

- Clasa **Resources**
  - Metoda **getResources()** din clasa **Context**
- Metode dedicate
  - **getString**()
  - **getColor**()
  - **getDimension**()
  - etc.
- Parametru: identificatorul resursei
  - clasa **R** sau clasa **android.R**

Clasa R



Clasa R

- Subclase
  - color
  - string
  - dimen
  - array
  - layout
  - id
  - menu
  - etc.
- Membri
  - Identificatori/denumire
- Exemple
  - R.**id**.*buton*
  - R.**string**.*denumire_aplicatie*
  - R.**color**.*rosu*

Referirea resurselor din cod

```
//preluarea unui sir de caractere
String sir = getResources().getString(R.string.mesaj);
```

```
//preluarea unei culori:
int culoare =  getResources().getColor(R.color.culoare_fundal);
```

```
//initializarea unei liste de valori din resurse
String [] optiuni=
    getResources().getStringArray(R.array.optiuni);
```

Selectori pentru configurații

Configurație	Semnificație	Exemple
port	Orientare verticală	layout-**port**
land	Orientare orizontală	layout-**land**
en, ro, es, en-rUK etc.	Codurile standard pentru limba utilizată și pentru regiune	values-en values-en-rUS values-ro
sw*VAL*dp	Lățimea minimă posibilă, indiferent de poziția ecranului	layout- sw320dp
h*VAL*dp	Înălțimea minimă disponibilă	layout-h720dp
w*VAL*dp	Lățimea minimă disponibilă	layout-w640dp
ldpi, mdpi, hdpi, xhdpi etc.	Densitatea ecranului	drawable-mdpi drawable-land-hdpi
small, normal, large, xlarge	Dimensiunea ecranului	layout-large
v14, v21 etc.	Versiunea ecranului	values-v14
mcc, mnc	Codul țării și al operatorului de telefonie mobilă	values-mcc266-mnc10 (Romania, Orange) values-mcc266 (Romania)

INTERFAȚA CU UTILIZATORUL

Interfața grafică

- Definită prin intermediul componentelor vizuale și a containerelor
- Componente vizuale (*widgets*): controale
- Componente de aranjare și structurare (**ViewGroup**/**Layout**): containere
- Asociată activităților și altor componente vizuale

Interfața grafică

- **Definire imperativă**
  - Fișiere XML
  - Separă codul de interfața grafică
- Definire declarativă
  - Jetpack Compose
  - Kotlin
- Definire procedurală
  - Codul sursă Java/Kotlin

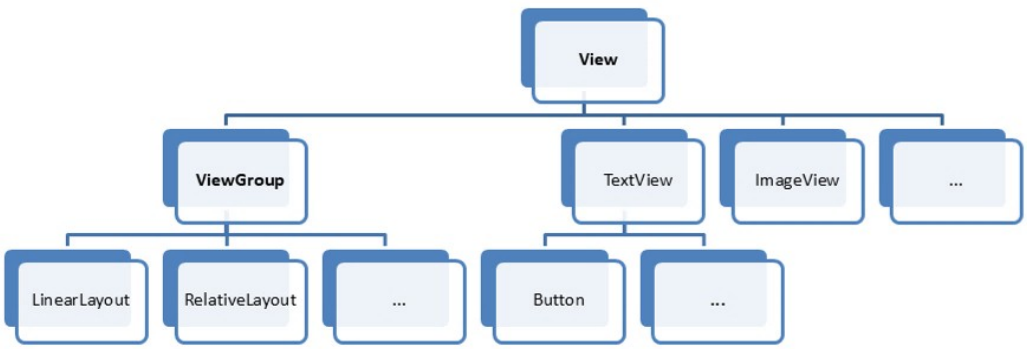
Interfața grafică

- Clasa **View**
  - Utilizată pentru desenarea suprafeței și tratarea evenimentelor
  - Clasa de bază pentru componente vizuale simple (widgets)
- Clasa **ViewGroup**
  - Derivată din clasa **View**
  - Clasă de bază pentru containere și componente vizuale complexe
- Pachetul android.view

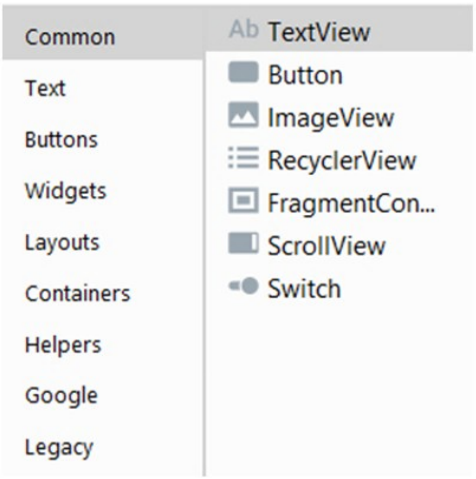
AndroidX

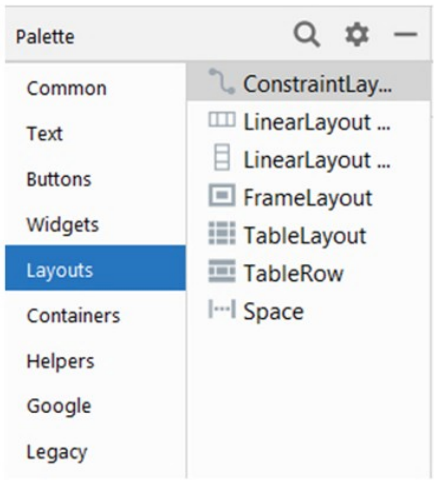
- Parte a Android Jetpack
  - colecție de biblioteci de clase
- Asigură compatibilitatea pentru versiunile anterioare Android
  - Fragmente, bara de acțiune, container tabular etc.
- Include componente noi
  - Ex: **ConstraintLayout, TextInputLayout, TextInputEditText, FloatingActionButton, ViewPager, DrawerLayout, RecyclerView, Snackbar** etc.
- Pachete **androidx**.*

Ierarhia de clase (componente)

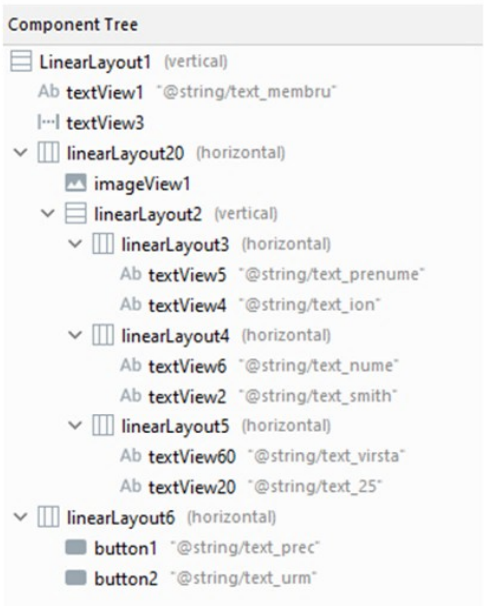


Paleta de componente





Exemplu structură interfață



INTERFAȚA ÎN MOD IMPERATIV

Interfața în mod imperativ

- Fișiere XML
- Definire
  - Elemente interfaţă grafică/elemente vizuale (res/layout)
  - Meniuri (res/menu)
- Posibilitatea de reutilizare (includere)
- Resurse referite
  - Valori constante (res/values)
- Asocierea componentelor vizuale activităților
  - metoda **setContentView()**

Fișiere XML (machete interfață)

- Descriu structura ecranelor sau a componentelor vizuale
- Fiecare componentă inclusă este reprezentată de un element XML și atributele asociate
- Elemente XML
  - Clase asociate componentelor (**LinearLayout**, **Button** etc.)
- Atribute
  - Proprietățile asociate componentelor (**id**, **layout_width**, **layout_height** etc.)

Proprietăți

- Atribute XML
- Domeniul de nume **android**
  - **xmlns:android="http://schemas.android.com/apk/res/android"**
- **android:***proprietate*
  - Specifice fiecărei componente vizuale
- **android:layout_***proprietate*
  - Clasa **LayoutParams**
  - Transmise componentei părinte în vederea poziționării și dimensionării

Valori atribute

- Valori constante
  - Șiruri de caractere
  - Dimensiuni
  - Culori
  - etc.
- Constante predefinite
- Resurse

Referirea resurselor (XML)

- @pachet:*tip_resursa/nume_resursa*
- Proiectul curent
  - *@tip_resursa/nume_resursa*
  - Exemplu:
    - **@dimen/dim_font**
    - **@color/culoare_fundal**
- Resurse predefinite
  - @android:*tip_resursa/nume_resursa*
  - Exemplu
    - **@android:color/background_light**
    - **@android:string/dialog_alert_title**

Proprietăți obligatorii

- **android:layout_width**
  - Lățime
- **android:layout_height**
  - Înălțime
- Dimensiuni: constante sau relative
- Valori posibile
  - **wrap_content** sau
  - **match_parent** sau
  - valori constante

Dimensiuni

- **px** (pixels)
- ***dp*** (density-independent pixels)
  - un dp = un pixel pentru o densitate de 160 dpi
- **mm**, **in**
  - Raportare la dimensiunea fizică a ecranului
- **pt** (puncte; 1/72 inch)
  - fonturi
- ***sp*** (scale-independent pixels)
  - fonturi

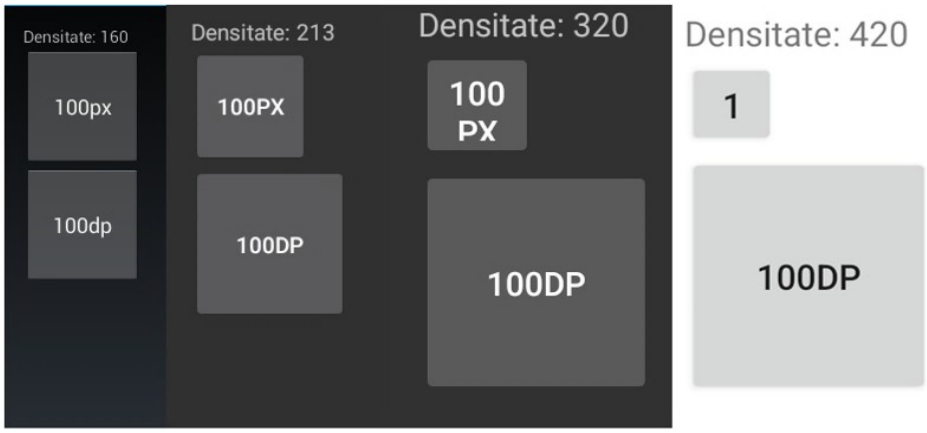
Densitatea (PPI/DPI)

- Densitatea $= \frac{\sqrt{hp^2+vp^2}}{diag}$
  - hp – rezoluția pe orizontală (pixeli)
  - vp – rezoluția pe verticală (pixeli)
  - diag – diagonala (inci)

Dimensiuni - conversii

- Din pixeli în dp
  - $dp = \frac{px * 160}{DPI}$
- Din dp în pixeli
  - $px = \frac{dp * DPI}{160}$

Dimensiuni



2.5”, 240x320 7”, 800x1280 4.7”, 768x1280 5”, 1080x1920

Identificarea componentelor

- Referirea componentei:
  - Poziționare/aliniere relativă
  - Modificare dinamică din cod
- **android:id**
- Asociere identificator: "@+id/identificator"
- Referire identificator existent:
  - XML: @id/identificator
  - Java/Kotlin: R.id.identificator

Referirea componentelor în cod

- Instanțiere pe baza identificatorului
- Metoda din clasa Activity:
  - View **findViewById**(int idResursa)
  - <T extends View> T **findViewById**(int idResursa)
- Exemplu
  - TextView tv = **findViewById**(R.id.tv_titlul);
  - tv.setText("Titlu modificat dinamic");
- Apelul se realizează **după invocarea** metodei **setContentView**()!

CONTAINERE

Containere

- Definesc structura vizuală a interfeței utilizator
- Structurarea conținutului vizual:
  - Liniară (verticală/orizontală)
  - Tabelară
  - Relativă
- Clasă de bază în ierarhie: **ViewGroup**

Containere

- **LinearLayout**
  - Orizontal sau vertical
- **ConstraintLayout**
  - Relativ la alte componente
- **TableLayout**
  - Linii si coloane
  - TableRow
- **FrameLayout**
  - Componentele vizuale sînt suprapuse (stivă)
- **ScrollView /HorizontalScrollView**
  - Permite derularea conținutului pe verticală/orizontală

Containere

- **RelativeLayout**
  - Relativ la alte componente
- **GridLayout**
  - Linii și coloane
  - Combină TableLayout cu LinearLayout
- **AbsoluteLayout** (depreciat)
  - Poziționare absolută

LinearLayout

- Modul de aranjare a componentelor: **android:orientation**
  - horizontal



  - vertical



LinearLayout

- Distribuirea spațiului rămas liber (proporții)
  - **android:layout_weight**

view1, *view2*, *view3*:
android:layout_width="wrap_content"

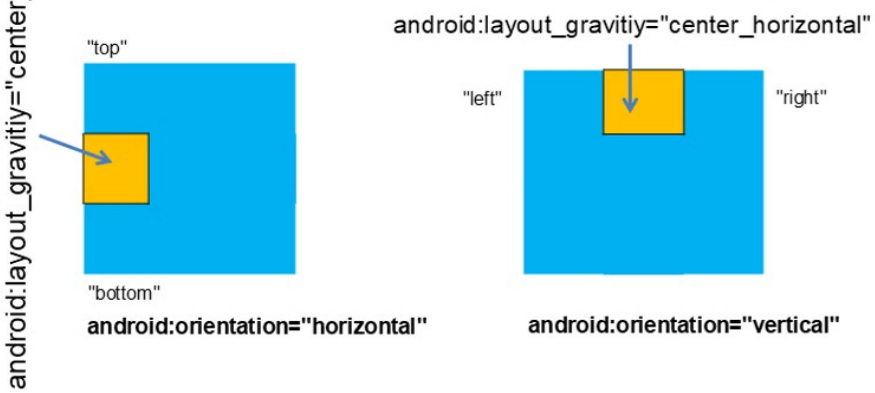
view1, *view2*, *view3*:
android:layout_width="0dp"
android:layout_weight="1"

view1, *view2*
android:layout_width="wrap_content"
view3:
android:layout_width="0dp"
android:layout_weight="1"

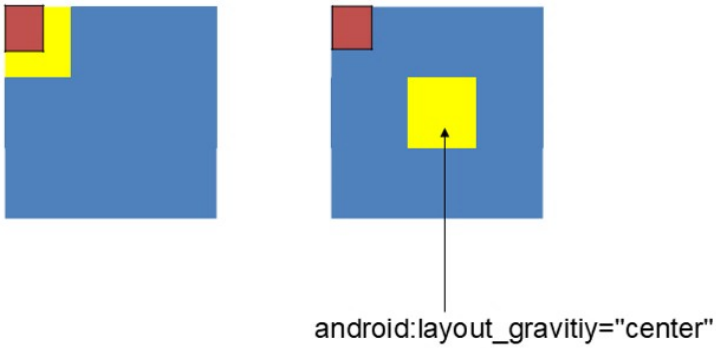


LinearLayout

- Alinierea componentelor în cadrul containerului
  - **android:layout_gravity**



FrameLayout



FrameLayout

- În mod uzual, este afișată o singură componentă vizuală
- Poziționarea componentelor:
  - Implicit: colțul stînga-sus
  - Suprapunere pe niveluri pe axa z
    - adîncime
- **android:layout_gravity**
  - Suprapunerea componentelor vizuale

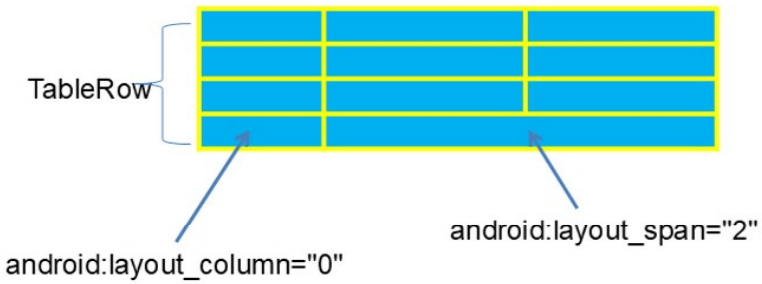
ScrollView/HorizontalScrollView

- Derivată din **FrameLayout**
- Acceptă o singură componentă vizuală;
- Afișare bare de defilare
  - android:**scrollbars**=*"vertical"*
- Ocuparea întregii suprafețe disponibile
  - android:**fillViewport**="true"

TableLayout

- Derivată din **LinearLayout**
- Linii
  - **TableRow**
- Coloana asociată
  - **android:layout_column**
- Întinderea pe mai multe coloane:
  - **android:layout_span**
- Adaptarea la conținut (pe baza indecșilor coloanelor)
  - android:**shrinkColumns**
    - Eliminarea spațiului liber
  - android:**stretchColumns**
    - Extinderea coloanelor pentru a ocupa spațiul liber
  - android:**collapseColumns**
    - Ascunderea coloanelor

TableLayout



ConstraintLayout

- Ierarhie care nu implică imbricarea controalelor
- Asemănător containerului de tip **RelativeLayout**
- Integrat cu editorul vizual din Android Studio
- Creșterea performanțelor

ConstraintLayout

```
dependencies {
   implementation 'androidx.constraintlayout:constraintlayout:x.x.x''
}
```

Constrîngeri

- Puncte de conectare între controale și
  - alte controale
  - containerul părinte
  - puncte de referință

Constrîngeri

- Poziționare relativă
  - Identificatorii controalelor sau **parent**
- Margini
  - Inclusiv controlul față de componentele ascunse
- Centrare/poziționare proporțională
- Înlănțuire
- Dimensiuni
- Puncte de referință

Poziționarea relativă

- **app:layout_constraint**[PunctSursă]_[PunctDestinație]= "[IdentificatorDestinație]"
- PunctSursă
  - Start
  - Top
  - Bottom
  - End
- PunctDestinție
  - toStartOf
  - toEndOf
  - toBottomOf
  - toTopOf

Înlănțuiri

- Proprietăți
  - **layout_constraintHorizontal_chainStyle**
  - sau **layout_constraintVertical_chainStyle**
- Se aplică primului element din înlănțuire
- Valori
  - spread
  - spread_inside
  - packed

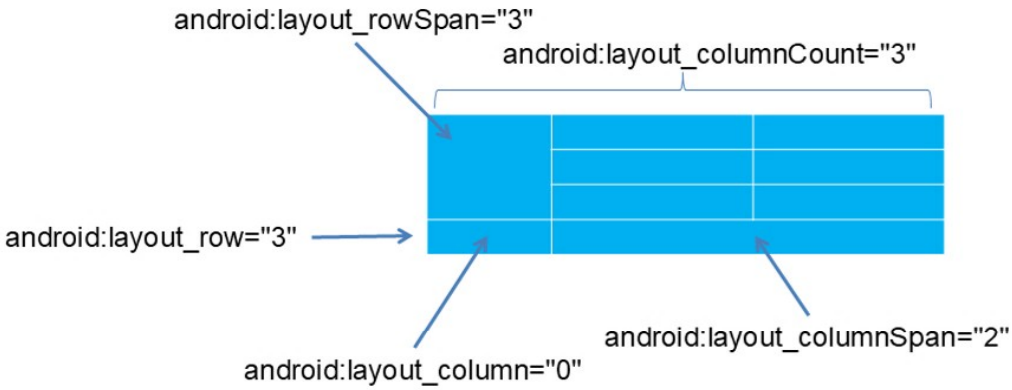
Manipulatori

- Dimensionare 
- Constrîngeri laturi 
- Constrîngeri puncte de referință 

GridLayout

- Număr coloane
  - android:**columnCount**
- Întinderea pe mai multe linii:
  - android:**layout_rowSpan**
- Întinderea pe mai multe coloane:
  - android:**layout_columnSpan**

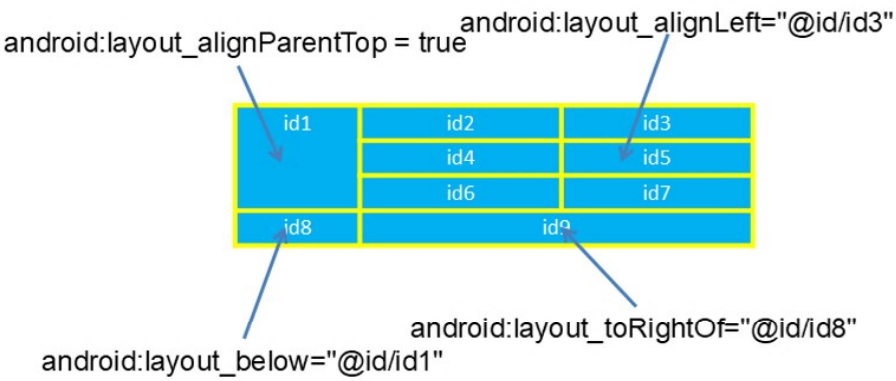
GridLayout



RelativeLayout

- Aliniere
  - Relativă la container (true/false):
    - **android:layout_alignParent***Top*|*Left*|*Right*|*Bottom*
    - **android:layout_center***Horizontal*|*Vertical*
    - **android:layout_centerInParent**
  - Relativă la alte componente (id-ul):
    - **android:layout_align***Left*|*Right*|*Start*|*Top*|*End*
- Poziționare
  - **android:layout_below**, **android:layout_above**
  - **android:layout_to***End*|*Left*|*Start*|*Right***Of**

RelativeLayout



COMPONENTE VIZUALE

Componente vizuale

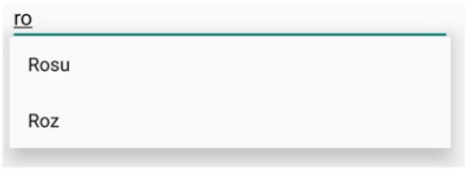
- Controale
  - Text (static, editabil)
  - Butoane
  - Liste
  - Bare de progres
  - etc.
- Simple și complexe
- Pachetul **android.widget**.*;

Afișare/preluare text

- **TextView**
  - Text static
- **EditText**
  - Introducere text
  - Suport pentru selecția tipului de date (**inputType**)
- **AutoCompleteTextView**
  - Derivată din EditText
  - Introducere text
  - Are asociată o listă de sugestii

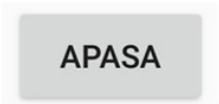






Butoane

- **Button**
  - Buton standard
  - Derivat din **TextView**
- **RadioButton**
  - Buton utilizat în selecția exclusivă
  - Derivat din **TextView**
  - Uzual, este inclus într-un **RadioGroup**
- **CheckBox**
  - Buton utilizat în selecția multiplă
- **ToggleButton**
  - Buton cu două stări, starea apăsat fiind marcată prin intermediul unui indicator
- **Switch**
  - Buton cu două stări (stînga-dreapta)











Imagini

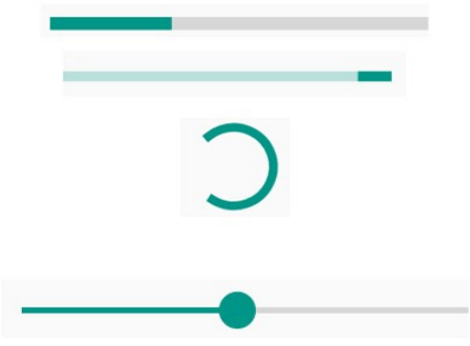
- **ImageView**
  - Control pentru afișarea de imagini
- **ImageButton**
  - Buton care permite afișarea unei imagini
  - Derivat din **ImageView**





Progres/selecție interval

- **ProgressBar**
  - Bară de progres
  - Determinată (limita superioară stabilită)
  - Indeterminată (fără limită superioară, ciclică)
- **SeekBar**
  - Derivat din **ProgressBar**,
  - permite selectarea valorii curente



Timp

- **AnalogClock**
  - Ceas analogic
- **TextClock**
  - DigitalClock (depreciată)
  - Ceas digital
  - Afișat conform setărilor de sistem
- **Chronometer**
  - Cronometru



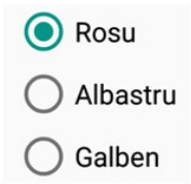
21:46

00:00

Grupare, spațiere

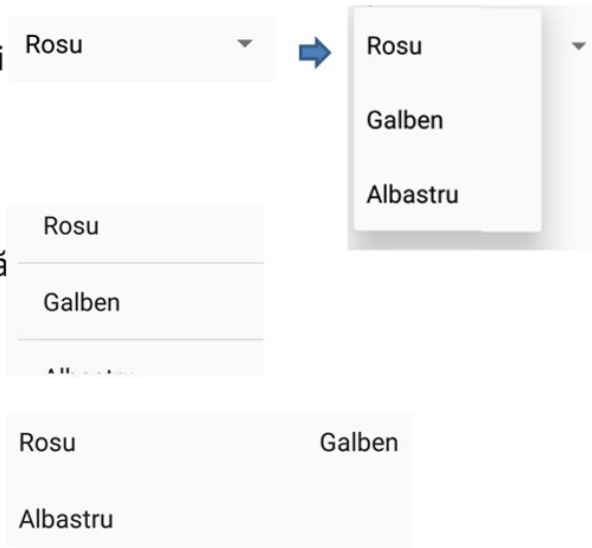
- **RadioGroup**
  - permite gruparea butoanelor radio
- **Space**
  - Inserarea de spații între componente





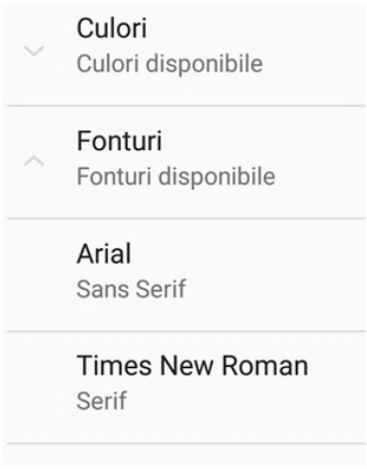
Liste

- **Spinner**
  - control asemănător unei liste combinate
- **ListView**
  - listă compusă care afișează elementele componente pe verticală
- **GridView**
  - permite poziționarea componentelor vizuale sub formă de linii și coloane



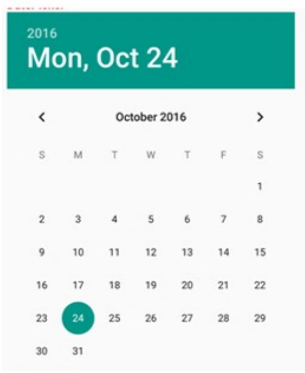
Liste

- **ExpandableListView**
  - listă compusă în care elementele sînt grupate pe categorii

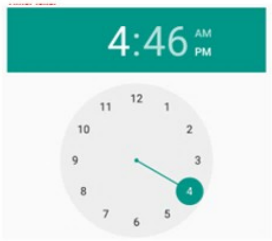


Selecție dată/oră

- **DatePicker**
  - selecția datei
  - derivat din **FrameLayout**



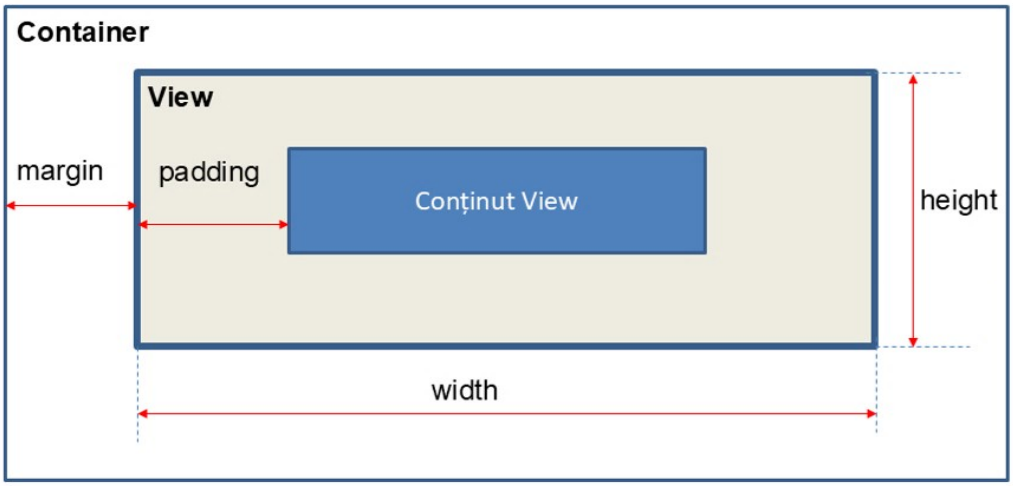
- **TimePicker**
  - selecția orei
  - derivat din **FrameLayout**



Proprietăți uzuale

Proprietate	Atribut	Exemple de valori
Textul asociat resursei	android:text	"Text în clar" @string/text_control
Distanţa conţinutului faţă de margini	android:padding android:padding*Top*|*Start*|*End*|*Bottom*	10dp
Culoarea de fundal	android:background	#FF00FF @drawable/imag
Alinierea conţinutului	android:gravity	center, left
Alinierea în cadrul containerului	android:layout_gravity	center, left
Distanţa faţă de marginile containerului	android:layout_margin, android:layout_margin*Top*|*Left*|*Right*|*Bottom*	30dp
Afișarea/ascunderea componentei	android:visibility	visible, invisible, gone

Spațierea



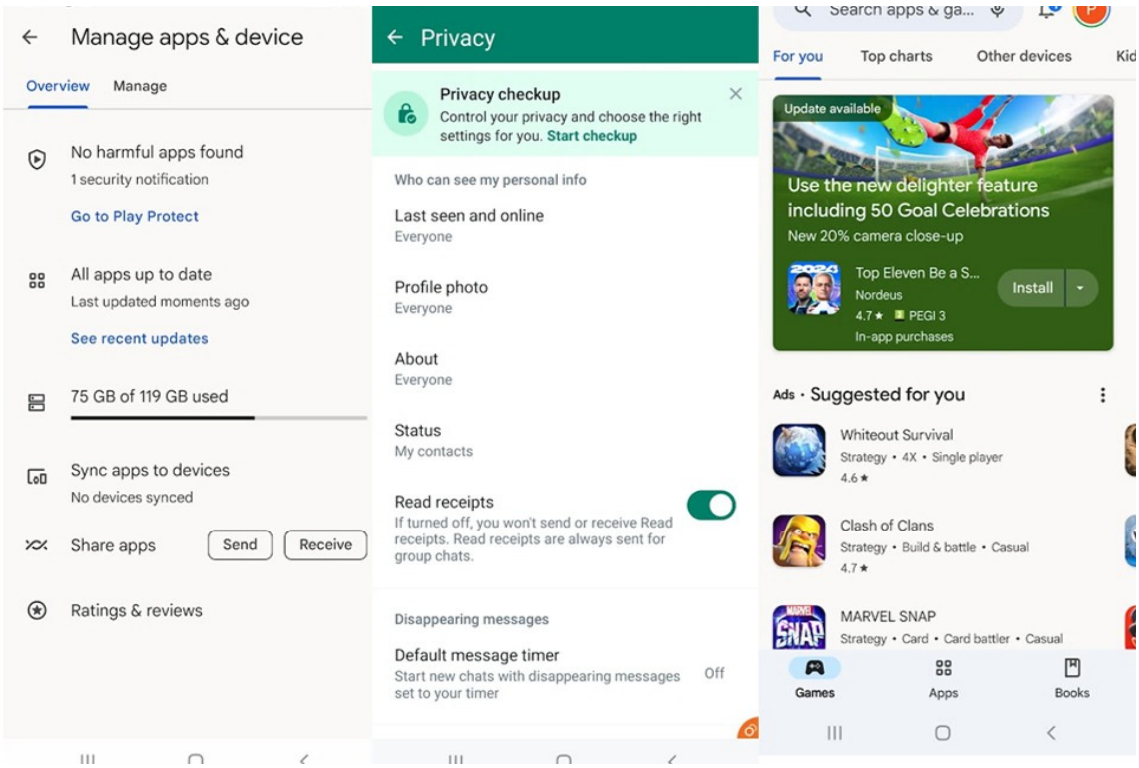
Exemplu

```
<!—res/layout/activity_main.xml-->
<LinearLayout <!-- ... -->
  android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:orientation="vertical" >
  <TextView
      android:id="@+id/tv_titlul"
      android:layout_width="wrap_content"
      android:layout_height="20dp"
      android:text="@string/titlul" />
  <EditText
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:ems="10"
      android:hint="@string/introduceti" />
  <Button
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/trimite" />
</LinearLayout>
```

```
<!-- /res/values/strings.xnl -->
<resources>
      <string name="titlul">Titlul</string>
      <string name="introduceti">Introduceti titlul</string>
      <string name="trimite">Trimite</string>
</resources>
```

```
// /src/Activitate.java
//in metoda onCreate()
setContentView(R.layout.activity_main)
```





EXEMPLE DISPOZITIV ANDROID

INTERFAȚA ÎN MOD IMPERATIV

INTERFAȚA ÎN MOD PROCEDURAL

Interfața în mod procedural

- Controalele inițializate din cod
- Constructori
  - Contextul curent
- Inițializare proprietăți
- Adăugare la containere
  - Metoda **addView**()
- Anumite dimensiuni trebuie calculate
- Asociere conținut în activitate
  - **setContentView**()
  - Poate fi un container sau un control

Proprietăți comune

- Dimensiuni
  - **getWidth**()
  - **getHeight**()
- Aspect
  - **setBackgroundColor**(int culoare)
  - **setBackground**(Drawable)
  - **setBackgroundResource**(int idResursa)

Proprietăți comune

- Spațiere (distanţă faţă de conţinut)
  - **setPadding**()
  - **getPaddingLeft**()
  - **getPaddingTop**()
  - **getPaddingRight**()
  - **getPaddingBottom**()
- Activare:
  - **setEnabled**(boolean)
  - **isEnabled**()
- Posibilitatea de selectare
  - **setFocusable**(boolean)
  - **isFocusable**()

Proprietăți comune

- Parametri container
  - **setLayoutParams**(LayoutParams)
  - LayoutParams **getLayoutParams**()
- Vizibilitate:
  - **setVisibility**(int)
    - valori: VISIBLE, INVISIBLE, GONE

Proprietăți comune

- Poziție (pixeli)
  - Absolută
    - setX()/getX()
    - setY()/getY()
  - Relativă la părinte
    - setLeft()/getLeft()
    - setTop()/getTop()
    - setRight()/getRight()
    - setBottom()/getBottom()

Proprietăți comune

- Tratare evenimente
  - **setOnClickListener**()
  - **setOnLongClickListener**()
  - **setOnDragListener**()
  - **setOnKeyListener**()
  - **setOnTouchListener**()

Dimenisiuni – conversii (în cod)

```
DisplayMetrics dm =
        getResources().getDisplayMetrics();

//conversie la pixeli
int px = (int) TypedValue.applyDimension(
        TypedValue.COMPLEX_UNIT_DIP, dp, dm);

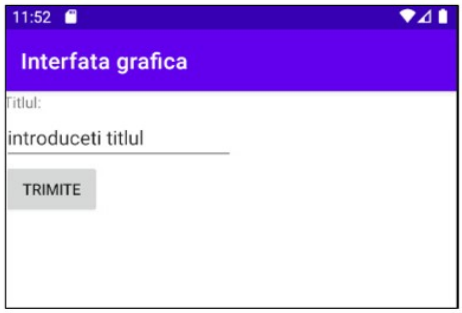
//sau

int px = Math.round(dp * dm.density);
```

EXEMPLU INTERFAȚA ÎN MOD PROCEDURAL

Exemplu Interfața în mod procedural

- Reproducerea ecranului
- Metoda **onCreate()** a unei activități
- Codul este imediat după apelul metodei **super.onCreate(*stare*)**



Interfața în mod procedural

```
//Inițializare container
LinearLayout layout = new LinearLayout(this);
layout.setOrientation(LinearLayout.VERTICAL);

//Adăugare control static
TextView tv = new TextView(this);
tv.setText("Titlul:");

//Conversie 20dp la pixeli
DisplayMetrics displayMetrics = getResources().getDisplayMetrics();
int hPixeli = (int)
TypedValue.applyDimension(TypedValue.COMPLEX_UNIT_DIP, 20,
displayMetrics);
```

Interfața în mod procedural

```
//Dimensiuni: înălțime 20dp, latime WRAP_CONTENT
LayoutParams lpt = new
LayoutParams(LayoutParams.WRAP_CONTENT, hPixeli);
tv.setLayoutParams(lpt);

//Dimeniusni: WRAP_CONTENT (pentru controlul de
//editare și buton)
LayoutParams lp = new
LayoutParams(LayoutParams.WRAP_CONTENT,
LayoutParams.WRAP_CONTENT);
```

Interfața în mod procedural

```
//Controlul de editare
EditText et = new EditText(this);
et.setText("introduceti titlul" );
et.setEms(10);
et.setLayoutParams(lp);

//Butonul
Button buton = new Button(this);
buton.setText("Trimite");
buton.setLayoutParams(lp);

//Adaugarea componentelor la container
layout.addView(tv);  layout.addView(et);  layout.addView(buton);

//Asocierea continutului
setContentView(layout);
```

Parametrii containerelor (cod)

- Clasa de bază **ViewGroup.LayoutParams**
- Definită în fiecare clasă de tip container
  - LinearLayout.LayoutParams, RelativeLayout.LayoutParams
- Referire din clasa View
  - **setLayoutParams**()
  - **getLayoutParams**()
- Proprietăți comune
  - width, height
- Proprietăți specifice containerului
  - gravity, weight
- Constructor
  - width, height

Parametrii containerelor

```
//inaltime 20 pixeli
LinearLayout.LayoutParams lp = new
LinearLayout.LayoutParams(LayoutParams.WRAP_CONTENT, 20);
lp.setMargins(10, 10, 10, 10);//left, top, right, bottom (pixeli)
view.setLayoutParams(lp);

//daca obiectul view este deja atasat la un container
LayoutParams lpb = view.getLayoutParams();
lpb.width = ViewGroup.LayoutParams. MATCH_PARENT;
lpb.height = ViewGroup.LayoutParams.WRAP_CONTENT;
view.setLayoutParams(lpb);
```

TRATAREA EVENIMENTELOR

Tratarea evenimentelor

- Interfețe de tip *listener*
  - Ex: **View.OnClickListener**
- Metode pentru tratarea evenimentelor
  - Ex.: **onClick()**
- Înregistrarea evenimentelor
  - Ex.: **setOnClickListener()**

Tratarea evenimentelor

Interfața	Metoda pentru tratarea evenimentului	Înregistrarea obiectului
View.OnClickListener	onClick()	setOnClickListener()
View.OnKeyListener	onKey()	setOnKeyListener()
View.OnTouchListener	onTouch()	setOnTouchListener ()
View.OnLongClickListener	onLongClick()	setOnLongClickListener()

Tratarea evenimentelor

- Implementarea interfeței într-o clasă
  - dedicată
  - existentă (clasa de tip activitate etc.)
- Implementarea interfeței într-o clasă anonimă
- Varianta rapidă pentru evenimentul *click*:
  - proprietatea **onClick** în fișierul XML
  - metoda de tratare a evenimentului în clasa activitate asociată

Tratarea evenimentelor (1)

```
public class Activitate extends Activity implements
                                        View.OnClickListener {

    public void onCreate(Bundle savedInstanceState) {

      //înregistrare clasă tratare eveniment pentru Button buton
      buton.setOnClickListener(this);
      //...
    }

    //tratare eveniment
    public void onClick(View view) {
        // Tratarea evenimentului
    }
}
```

Tratarea evenimentelor (2)

```
public class EvenimClick implements View.OnClickListener {
        //tratare eveniment
        public void onClick(View view) {
                // Tratarea evenimentului
        }
}
public class Activitate extends Activity {
        public void onCreate(Bundle savedInstanceState) {
                //înregistrare clasă tratare eveniment pentru Button buton
                buton.setOnClickListener(new EvenimClick());
                //...
        }
}
```

Tratarea evenimentelor (3)

```
//Înregistrare și implementare clasă anonimă
//pentru tratare eveniment pentru controlul
//buton de tip Button. Codul poate fi în onCreate()

buton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
                // Tratarea evenimentului
        }
});
```

Tratarea evenimentelor (3.1)

```
/*Înregistrare și implementare clasă anonimă și
o funcție anonimă (expresie Lambda) pentru
tratare eveniment pentru controlul buton de tip
Button. Codul poate fi în metoda onCreate()*/

button.setOnClickListener(view -> {
   //Tratarea evenimentului
});
```

Tratarea evenimentelor (4)

- XML
  - android:onClick="**numeMetoda**"
- În clasa derivată din Activity:
  - public void **numeMetoda**(View view) { … }
- Aceeași metodă pentru mai multe controale
  - Diferența între controale:
    - view.getId()
- Abordare nerecomandată

Bibliografie

- Android Developers, https://developer.android.com
- M. L. Murphy, The Busy Coder's Guide to Android Development, CommonsWare, https://commonsware.com/Android, 2019
- P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012
- P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015